Перед тем как начать работать с распознаванием образов, необходимо подготовить изображение: размыть, окрасить в серый, изменить размер и т.д. В зависимости от задачи подбираются подходящие функции оптимизации и трансформации.

Стандартные функции OpenCV

Пример 1: Загрузите изображение, поверните его на 45° вправо, нарисуйте красный круг и зелёный прямоугольник, добавьте надпись и сохраните результат.

Решение:

```
import cv2
import numpy as np

img = cv2.imread('image.jpg')
# Загрузка изображения
# Метод imread загружает изображение в формате BGR (не RGB!)

rows, cols = img.shape[:2]
# Получаем размеры изображения (количество строк и столбцов)
# img.shape возвращает кортеж (height, width, channels), нам нужны только
height и width

# Поворот на 45 градусов

rotation_matrix = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
rotated_img = cv2.warpAffine(img, rotation_matrix, (cols, rows))
# Создаем матрицу поворота для вращения изображения на 45 градусов
# Аргументы функции getRotationMatrix2D():
# ((cols/2, rows/2)) - точка вращения (центр изображения)
# 45 - угол поворота в градусах (положительное значение - поворот против
часовой стрелки)
# 1 - коэффициент масштабирования (1 означает сохранение оригинального размера)
# Применяем аффинное преобразование (вращение) к изображению
# warpAffine выполняет поворот изображения с использованием полученной матрицы
поворота
# Третий аргумент (cols, rows) задает размер выходного изображения
rotated_img = cv2.warpAffine(img, rotation_matrix, (cols, rows))


# Рисуем фигуры
cv2.circle(rotated_img, (100, 100), 50, (0, 0, 255), -1)  # Красный круг
```

- **(100, 100)** — координаты центра окружности (левый верхний угол изображения считается точкой (0, 0)).
- **50** — радиус окружности.
- **(0, 0, 255)** — цвет окружности в формате BGR (Red — красный цвет).
- **-1** — толщина линии (значение "-1" означает заливку фигуры целиком).

```
cv2.rectangle(rotated_img, (200, 200), (300, 300), (0, 255, 0), 3)  # Зеленый
прямоугольник
```

- **(200, 200)** — координаты левого верхнего угла прямоугольника.
- **(300, 300)** — координаты правого нижнего угла прямоугольника.
- **(0, 255, 0)** — цвет прямоугольника в формате BGR (Green — зелёный цвет).
- **3** — толщина контура прямоугольника (3 пикселя).

```
# Надпись
cv2.putText(rotated_img, 'Hello!', (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,
255, 255), 2)
```

📄 **(50, 50)** — координаты расположения текста (верхний левый угол надписи).Координаты начинаются с верхнего левого угла изображения.
📄 **cv2.FONT_HERSHEY_SIMPLEX** — стиль шрифта. Это стандартный шрифт OpenCV, простой и понятный.
📄 **1** — размер шрифта (чем больше значение, тем крупнее текст).
📄 **(255, 255, 255)** — цвет текста в формате BGR (в данном примере — белый цвет).
📄 **2** — толщина линии текста (толщина контура символов).

```
# Сохраняем результат
cv2.imwrite('output.jpg', rotated_img)
```

Задание 3.

Взять изображение дерева зимой. Используя OpenCV, напишите программу, которая:

1. Загружает изображение.
2. Отображает его вокруг вертикальной оси.
3. Нарисуйте солнечный круг золотистого цвета на небе (в верхнем правом углу).
4. Добавьте снег (белые точки) случайным образом по всей картинке.
5. Выведите строку «Happy Winter!» красными большими буквами внизу изображения.
6. Добавьте зелёный круглый шар с надписью «NEW YEAR» на ветке дерева.
7. Сохраните итоговое изображение в файл new_year_tree.jpg.
8. Отобразить исходное и итоговое изображения

⚙ Требования к реализации:

- Используйте библиотеку OpenCV для загрузки и модификации изображения.
- Круг солнца должен быть золотистым (желательно подобрать естественный цвет).
- Белый снег разместите случайно и равномерно по всему изображению.
- Надпись «Happy Winter!» должна располагаться ровно снизу и занимать всю ширину изображения.
- Красивый новогодний шар на дереве можно оформить ярким зеленым цветом.

Сложение изображений (добавление шума)

Пример 2: Суммируйте два изображения и добавьте гауссов шум.

Решение:

```
import cv2
import numpy as np

# Загружаем два изображения
img1 = cv2.imread('image1.jpg')
img2 = cv2.imread('image2.jpg')

# Суммируем изображения
summed_img = cv2.addWeighted(img1, 0.5, img2, 0.5, 0)
```

- **0.5** — вес первого изображения. То есть вклад первого изображения в итоговый результат составляет половину.
- **img2** — второе изображение.
- **0.5** — вес второго изображения. Аналогично первому, вклад второго изображения также составляет половину.
- **0** — дополнительный скалярный сдвиг (gamma), в данном случае равен нулю.

```
# Добавляем гауссов шум
mean = 0
var = 100
sigma = var ** 0.5
noise = np.random.normal(mean, sigma, summed_img.shape).astype(np.uint8)
noisy_img = cv2.add(summed_img, noise)
```

- **mean = 0** — среднее значение распределения шума (математическое ожидание). Устанавливаем на 0, так как чаще всего нужен симметрично распределённый шум.
- **var = 100** — дисперсия шума. Величина, характеризующая разброс значений шума. Чем больше дисперсия, тем сильнее шум проявляется на изображении.
- **sigma = var ** 0.5** — стандартное отклонение (σ). Равно корню из дисперсии (по определению стандартного отклонения σ=dispersion$^{1/2}$).

```
# Показываем результат
cv2.imshow('Noisy Summed Image', noisy_img)
cv2.waitKey(0)
```

- **waitKey()** — функция ожидает нажатия клавиши клавиатуры.

- **0** — бесконечная пауза, пока не нажата любая кнопка. Если поставить положительное число (например, 1000), окно будет открыто указанное количество миллисекунд.

Задача 4:

Создайте реалистичный эффект старения снимка

Задание:

Представьте, что у вас есть фотография старинного дома/объекта, сделанная недавно. Нужно превратить её в фотографию, выглядящую так, будто ей сто лет. Вам предстоит имитировать эффекты старения бумаги и ухудшение качества изображения, добавив следующие особенности:

1. Немного уменьшите насыщенность цветов (сделайте снимок слегка блеклым).
2. Добавьте виньетку (затенение по краям).
3. Наложите лёгкое зернистое зерно (эффект плёнки прошлого столетия).
4. Дополните изображение несколькими царапинами и пятнами, как бывает на старых снимках.
5. Осторожно осветлите середину изображения, придав ощущение старого пожелтевшего фона.

Подсказки:

1. **Изменение насыщенности цветов:**
   - Воспользуйтесь преобразованием изображения в цветовое пространство HSV (cv2.cvtColor()). Уменьшите компонент V (яркость) и S (насыщенность), оставив H (тон) без изменений.
2. **Создание виньетки (затенение по краям):**
   - Создайте маску с формой круга, где центр яркий, а края затемнены. Примерьте маску к изображению, умножив изображение на маску.
3. **Имитация зерна плёнки (зернистый шум):**
   - Добавьте слабый случайный шум (например, гауссов шум) к изображению с помощью функции np.random.randn() или cv2.addWeighted().
4. **Эффект царапин и пятен:**
   - Случайно размещайте белые пятна и тонкие линии (например, с помощью функций рисования, таких как cv2.line() и cv2.ellipse()).
5. **Осветление середины изображения:**
   - Используйте градиентную маску, которая плавно повышает яркость в центральной зоне изображения.
6. Отобразить исходное и итоговое изображения, маску

Усреднение изображений (удаление шума)

Пример 3: Удалите шум с изображения с помощью усредняющего фильтра.

Решение:

```
import cv2

# Загружаем зашумлённое изображение
noisy_img = cv2.imread('noisy_image.jpg')

# Применяем усредняющий фильтр
blurred_img = cv2.blur(noisy_img, (5, 5))

# Показываем результат
cv2.imshow('Blurred Image', blurred_img)
cv2.waitKey(0)
```

Задача 5

Помогите художнику восстановить старую картину!

Представьте, что вы нашли старое полотно, покрытое слоем пыли и повреждений. Картина сохранилась плохо, и нужно вернуть ей первоначальный облик, очистив изображение от шума и следов повреждения.

Задание:

Используя OpenCV, восстановите поврежденную картину художника. Для этого:

1. Удалите сильные шумы с помощью усредняющего фильтра.
2. Устраните мелкие дефекты (царапины и следы грязи) с помощью морфологических операций (например, дилатации и эрозии).
3. Верните картине её изначальную красоту, сохранив оригинальные детали и краски.
4. Отобразить исходное и итоговое изображения

🎯 Условия:

- Начальное изображение сильно зашумлено и испорчено мелкими дефектами.
- Используйте фильтры средней величины (размер окна ~ 5×5).
- Оценивайте результат визуально, обращая внимание на сохранность мелких деталей.

📊 Подсказки:

Примените усредняющий фильтр с окном небольшого размера (~5×5 пикселей) с помощью функции **cv2.blur(src, kernel_size)**
Используйте морфологические операции для очистки мелких повреждений:

Сначала примените оператор эрозии (**cv2.erode(src, kernel)**) для уменьшения областей дефекта.
Затем примените оператор дилатации (**cv2.dilate(src, kernel)**) для сглаживания оставшихся артефактов.